

```

class LidarMeasure:
    def __init__(self, angle, distance):
        self.angle = angle
        self.distance = distance
        print("angle=" + str(angle))
        print("distance=" + str(distance))

    def __repr__(self):
        if int(self.angle) <= 220:
            #print("angle=" + str(self.angle))
            print("Safe distance=" + str(self.distance))
            if int(self.distance) <= 300:
                if int(self.distance) != 0.0:
                    print("very close please slow speed!!!!!!!!!!!!!!!!!!!!")
                    print("angle=" + str(self.angle))
                    print("distance=" + str(self.distance))
            #print("grater")
            return "< a"grater" < a" + str(self.angle) + " " + str(self.distance) + "mm"

class LidarX2:
    def __init__(self, port):
        self.port = port
        self.baudrate = 115200
        self.connected = False
        self.measureThread = None
        self.stopThread = False
        self.measureList = []
        self.serial = None

```

Fig. 7: Code extraction and the lidar data

```

import time
from LidarX2 import LidarX2

lidar = LidarX2("/dev/ttyUSB0") # Name of the serial port, can

if not lidar.open():
    print("Cannot open lidar")
    exit(1)

t = time.time()
while time.time() - t < 20: # Run for 20 seconds
    measures = lidar.getMeasures()
    reading = str(measures) # Get latest lidar measures
    parts = reading.split(":")
    print(parts)
    #first_value = parts[0]
    #second_value = parts[1]
    #print("First value:", first_value)
    #print("Second value:", second_value)

    time.sleep(1)

lidar.close()

```

Fig. 8: Code setting port name and baud rate

As can be seen in Fig. 4, YDLIDAR X2 has four pins. Pins VCC and GND are for connecting power, pin TX is for transmitting the data, pin M_CTR is used for the signal to control the motor. Pin M_CTR further connects to another port that provides you the USB to connect with your system. So you need to simply connect the lidar with the RPi board using USB plug.

Important things to keep in mind while mounting a lidar are vibration and stabilisation. The data points get wrong if the lidar vibrates too much, so it needs to be mounted with a rubber pad or some kind of shock-absorbing material.

Sensing angle also plays an important role. Generally, we fix the lidar so that it can sense what's in front and behind a vehicle, if its covering range is 245 or 180 degrees. But while using 360-degree lidar, it should be mounted as per its Zero point, as shown in Fig. 5 and Fig. 6.

Software

For coding and getting the data for this anti-collision system, you need a spectral beam combining SBC using NVIDIA Jetson RPi. Install the Python module, lidar SDK, and libraries according to the lidar being used. If you are using YDLIDAR X2, install the library for the same. Next, install the matplotlib and numpy library to do mathematical calculations to find the distance and angle data using the lidar data.

To install the Python modules, open terminals and run following:

```

sudo pip3 install numpy
sudo pip3 install matplotlib

```

Now clone the library of YDLIDAR X2 as per <https://github.com/nesnes/YDLidarX2>

Create the code to get the data

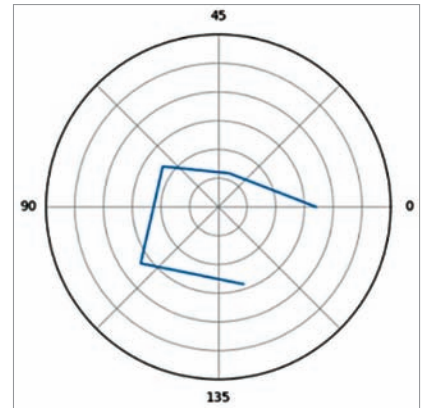


Fig. 9: Lidar mapping the surroundings

from lidar. The lidar library has YDLIDAR.py file, which should be opened to prepare the code to send and view the angle and distance data. Make an if condition to check the angle and distance of obstacle. To keep the front of vehicle at safe distance, use the angle range of around 245 degrees.

To keep the vehicle away at safe distance from the other things on all sides, set the desired safe distance in if condition. You can change and tweak the safe distance in if condition as per your requirement. Under the if condition you set the GPIO as output to alert, or apply breaks automatically, or simply print the alert to the Python cell asking you to drive slowly to avoid an accident. Fig. 7 shows code extraction and the lidar data

In the main code, you need to set the port name and baud rate. Set the port name according to the USB port name you connect the YDLIDAR to and the baud rate according to your lidar's baud rate. The lidar baud rate during testing of the author's prototype was 115200. Fig. 8 shows the code setting port name and baud rate.

If you want to see the surroundings captured by the lidar, you can add the code for it. Fig. 9 shows the lidar mapping the surroundings while Fig. 10 shows the code to get the map of surroundings using the